

Practica - 25 Evaluación Comparativa de Procesamiento Geoespacial

Luis Gálvez

Tabla de contenidos

1 Resultados de la comparativa	1
1.1 Taller	1
1.1.1 Desarrollo	1
1.2 Preguntas de análisis	2
1.2.1 Desarrollo	2

1 Resultados de la comparativa

1.1 Taller

- Indiquen en `respuestas.qmd`:
 1. Qué opción usaron
 2. Los tiempos obtenidos
 3. Si notaron diferencias frente a JupyterLab o VSCode

1.1.1 Desarrollo

1. Se ejecutó de manera directa en powershell con la expresión `docker exec contenedor_sig_unal Rscript + ruta` (o adaptado segun corresponda)

Comparativa Benchmark	Stars	Terra	Rasterio	Raster
Jupyter Notebook				
Tiempo de ejecución (Segundos)	12.679	14.143	2.375	7.602
Promedio	3766.625	3766.625	3766.62463	3766.62463
Vs Code				
Tiempo de ejecución (Segundos)	11.053	12.508	2.362	7.017
Promedio	3766.625	3766.625	3766.62463	3766.62463
PowerShell				
Tiempo de ejecución (Segundos)	9.426	10.873	2.348	6.432
Promedio	3766.625	3766.625	3766.62463	3766.62463

- 2.
3. La diferencia fue relativamente poca, aunque en parte se debe a los procesos en segundo plano y el espacio en disco del que disponía la máquina.

1.2 Preguntas de análisis

1. Entorno de ejecución
¿Notaron diferencias de tiempo entre:
 - JupyterLab
 - VSCode (terminal integrada)
 - Windows Terminal (PowerShell)

Den **una razón técnica posible** (overhead del kernel, entorno, proceso, etc.).

2. Abstracción en la práctica
¿En qué motor creen que el **costo de las abstracciones** es más visible?
Relacionen su respuesta con los tiempos observados.
3. Julia y el costo de compilación (Warm-up)
¿El efecto del *warm-up* de Julia se notó más en algún entorno específico?
Expliquen brevemente por qué.
4. Elección informada
Después de ejecutar el benchmark en **tres entornos distintos**,
¿cambiarían su elección del “Titán” para una emergencia ambiental real?

Justifiquen en **máximo 5 líneas**.

1.2.1 Desarrollo

1. PowerShell fue con diferencia la más rápida, probablemente por que debe correr menos procesos en segundo plano y no ocupa recursos en mantener una interfaz grafica compleja como VS o Jupyter, los cuales estuvieron bastante cerca en rendimientos (Aunque en VS hubo complicaciones debido a que contaba con poco espacio en disco y el computador estaba bastante lento, así que hubo que depurar algunas cosas y fue mas rapido), sin embargo jupyter es el más lento, probablemente por que debe mantener una comunicación constante con el servidor y debe mantener una interfaz más compleja, además de haber mas capas de por medio entre el ejecutor y el archivo.
2. Terra y stars fueron los más demorados, y probablemente los mas costosos a nivel computacional debido a que estos motores realizan más procesos internos de alto nivel antes de ejecutar la operación matemática simple

sobre cada píxel, mientras que rasterio opera más cerca a lenguajes como C, lo que lo hace más rápido.

3. Se notó más en jupyter debido a las capas intermedias, se debe principalmente a que Julia utiliza compilación Just-In-Time (JIT), es decir que la primera vez que se ejecuta una función, el lenguaje debe compilar parte del código antes de correrlo realmente, este proceso inicial añade tiempo extra a la primera ejecución.
4. Escogería rasterio por su eficiencia y tiempos de ejecución ya que su integración directa con GDAL y NumPy reduce el costo de abstracciones y acelera operaciones masivas pixel a pixel. Lo que en escenarios críticos toma especial relevancia, esa diferencia de rendimiento puede ser determinante para procesar imágenes satelitales rápidamente y apoyar la toma de decisiones de manera oportuna.